

MLP

- ↳ MLP is define most complex architecture of artificial neural network.
- ↳ It is formed from multiple layer of the perception.
- ↳ It has three layer: Input layer, hidden layer and output layer.
- ↳ If MLP.NN has more than one hidden layer then it is called a deep ANN.
- ↳ MLP-learn with i/p layer and propagate data forward to the output layer.
- ↳ Based on q_p , error is computed and try to minimize the error using back propagation.
- ↳ In backpropagation ~~weight is updated~~ derivative of ~~error~~ error with respect to weight is use to update model or train the model for multiple epochs.
- ↳ Finally trained model is used tested with test sample which gives final predicted class labels.

36

JUL 2018							AUG 2018						
M	T	W	T	F	S	S	M	T	W	T	F	S	S
30	31				1				1	2	3	4	5
2	3	4	5	6	7	8	6	7	8	9	10	11	12
9	10	11	12	13	14	15	13	14	15	16	17	18	19
16	17	18	19	20	21	22	20	21	22	23	24	25	26
23	24	25	26	27	28	29	27	28	29	30	31		

June 2018

Wednesday

20

wk 25 • day 171

MLP

X

- ↳ MLP are not ideal for processing patterns.
 - ↳ For sequential data, the RNN are ^{best} because their patterns allow the network to discover dependence on the historical data.
 - ↳ CNN used for classification, segmentation for data image. In some cases on CNN in the form of ConvID/ID is also used for network with sequential input data.
 - ↳ It consist of three layer - i/p, o/p & hidden.
 - ↳ In MLP, the data flows in the forward dirn from i/p to o/p layer.
 - ↳ neurons in MLP are trained with back-propagation.
 - ↳ The MLP are designed to approximate continuous function and can solve problems with which are not linearly.
 - ↳ Except for the i/p nodes, each node is a neuron that uses a non-linear activation function.
 - ↳ A perceptron (single perceptron) is linear classifier and produce a single o/p.
 - ↳ In MLP, train on a set of i/p - o/p pairs and learn to model the correlation b/w these i/p and o/p.
 - ↳ Training involves adjusting the parameters or weight or biases of model in order to
- The first step toward success is taken when you refuse to be a captive of the environment in which you first find yourself

June 2018

21

Thursday

Wk 25 • day 172

MAY 2018							JUN 2018						
M	T	W	T	F	S	S	M	T	W	T	F	S	S
7	8	9	10	11	12	13	4	5	6	7	8	9	10
14	15	16	17	18	19	20	11	12	13	14	15	16	17
21	22	23	24	25	26	27	18	19	20	21	22	23	24
28	29	30	31				25	26	27	28	29	30	

minimize error.

- 1) Backpropagation is used to make those weight and bias adjustments relative to the error, and the error itself can be measured in a variety of ways (RMSE)
- 2) In the forward pass, single flow moves from the input layer through the hidden layer to output layer, and
- 3) In backward pass, using backpropagation and chain rule of calculus partial derivatives of the error function w.r.t the various weights and biases are back-propagated through MLP. That act of differentiation gives us a gradient; as long as the parameter may be adjusted as they move the MLP one step closer to the error minimum.

4) Every node in multi-layer perceptron uses a sigmoid activation function. The sigmoid activation function takes real values as input and converts them to number between 0 and 1 using sigmoid formula.

$$\sigma = \frac{1}{1 + \exp(-x)}$$

Notes

If at first you don't succeed, failure may be your style

JUL 2018							AUG 2018						
M	T	W	T	F	S	S	M	T	W	T	F	S	S
30	31					1	6	7	8	9	10	11	12
2	3	4	5	6	7	8	13	14	15	16	17	18	19
9	10	11	12	13	14	15	20	21	22	23	24	25	26
16	17	18	19	20	21	22	27	28	29	30	31		
23	24	25	26	27	28	29							

June 2018

Friday

Wk 25 • day 173

22

Weight in MLP Neuron:

- ↳ weights on the inputs are very much like the coefficients used in regression equation.
- ↳ Like linear regression larger weights indicates increased complexity and fragility. It is desirable to keep weights in the n/w small and regularization techniques can be used.

- ↳ for example: a neuron may have two inputs in which case it requires ~~two~~ three weights. one for each ip and one for bias.
- ↳ weights are often initialized to small random values in range 0 to 0.3

BUT activation function:

- ↳ The weighted ip are summed and passed through an activation function sometime called a transfer function.

- ↳ An activation function is a simple mapping of summed weighted ip to the output of the neuron. It is called an activation function because it governs the

Notes

June 2018

23

Saturday

Wk 25 • day 174

MAY 2018							JUN 2018						
M	T	W	T	F	S	S	M	T	W	T	F	S	S
7	8	9	10	11	12	13	4	5	6	7	8	9	10
14	15	16	17	18	19	20	11	12	13	14	15	16	17
21	22	23	24	25	26	27	18	19	20	21	22	23	24
28	29	30	31				25	26	27	28	29	30	31

→ threshold at which the neuron is activated and strength of the output signal.
if of summed ip was above a threshold for example 0.5 then the neuron would output at 1.0 otherwise it would output a 0.

Epoch — x — x — x — x —

→ one round of updating the n/w for entire training dataset is called an epoch.

Weight Update:

→ Online learning: update from the errors calculated for each training example and then

→ Batch learning: The errors can be saved up across all the training examples and the network can be updated at the end. This is often more stable.

→ amount of weight updated is controlled by the a configuration parameters called the learning rate. It is also called the step size.

Learning rate Decay: It is used to decrease the learning rate over epochs to allow the n/w to make large changes to the weights at beginning and smaller fine tuning at the end.

Success is simple. Do what's right, the right way, at the right time.

2014

28.05.2014

MLP

WEDNESDAY

MAY

28

WK-22.148-217

error signal produced at output neuron

$$e_j(n) = d(n) - y_j(n) \quad \text{--- (1) (4.2)}$$

instantaneous error energy at neuron j

$$E_j(n) = \frac{1}{2} e_j^2(n) \quad \text{--- (2) (4.3)}$$

total instantaneous energy for whole net

$$E(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n) \quad \text{--- (3) (4.4)}$$

error energy average over the training sample

$$E_{av} = \frac{1}{N} \sum_{n=1}^N E(n)$$

$$= \frac{1}{2N} \sum_{n=1}^N \sum_{j \in C} e_j^2(n) \quad \text{--- (4) (4.5)}$$

backpropagation algorithm.

Induced local field $V_j(n)$

$$V_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n) \quad \text{--- (5) (4.6)}$$

output of neuron j at nth iteration

$$y_j(n) = \phi_j(V_j(n)) \quad \text{--- (6) (4.7)}$$

Mon	Tue	Wed	Thu	Fri	Sat	Sun
22	23	24	25	26	27	28
29	30	31	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

29

MAY

THURSDAY

M-22.149-216

29.05.2014

According to chain rule calculus

$$\frac{\partial E(n)}{\partial w_{ij}(n)} = \frac{\partial E(n)}{\partial e_j} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ij}(n)}$$

Differentiating both side of equation (4) with respect to $e_j(n)$ we get

$$\frac{\partial E(n)}{\partial e_j(n)} = e_j(n) \quad \text{--- (5) (4)}$$

Differentiating both side of equation (1) with respect to $y_j(n)$ we get

$$\frac{\partial e_j(n)}{\partial y_j(n)} = -1 \quad \text{--- (6) (4)}$$

Differentiating equation (6) with respect to $y_j(n)$ we get

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \phi(v_j(n)) \quad \text{--- (7) (4)}$$

Finally, differentiating equation (5) with respect to $w_{ij}(n)$ we get

Courage is a peculiar kind of fear.

MAY
2014
18
19 5 6 7 8 9
20 10 11 12 13 14
21 15 16 17 18 19
22 20 21 22 23 24
23 25 26 27 28 29
30 31

2014

30.05.2014

MAY

30

FRIDAY

WK 22.155-216

$$\frac{\partial \psi_j(n)}{\partial w_{ji}(n)} = y_i(n) \quad \text{--- (11) --- (4.12)}$$

(7) (4.3) Putting the equations (8), (9), (10) & (11) in equation (7) we get.

$$\frac{\partial E(n)}{\partial w_{ji}(n)} = -e_j(n) \phi'(\psi_j(n)) y_i(n) \quad \text{--- (12) (4.13)}$$

The correction $\Delta w_{ji}(n)$ applied to $w_{ji}(n)$ is defined by the delta rule

$$\Delta w_{ji} = -\eta \frac{\partial E(n)}{\partial w_{ji}(n)} \quad \text{--- (13) (4.14)}$$

Putting the equation (12) in equation (13) we get.

$$\Delta w_{ji} = \eta e_j(n) \underbrace{\phi'(\psi_j(n)) y_i(n)}_{\text{local gradient}} \quad \text{--- (14)}$$

$$\Delta w_{ji} = \eta s_j(n) y_i(n) \quad \text{--- (15) (4.15)}$$

Notes

where local gradient $s_j(n)$ is defined by

$$s_j(n) = \frac{\partial E(n)}{\partial \psi_j(n)}$$

$$= \frac{\partial E(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial \psi_j(n)} \cdot \frac{\partial \psi_j(n)}{\partial w_{ji}(n)}$$

The best way to escape from a problem is to solve it.

$$= e_j(n) \cdot \phi'(\psi_j(n)) \quad \text{--- (16) (4.16)}$$

JUN 2014								
WK	M	T	W	T	F	S	S	
22	.	.	.	.	.	.	.	1
23	2	3	4	5	6	7	8	
24	9	10	11	12	13	14	15	
25	16	17	18	19	20	21	22	
26	23	24	25	26	27	28	29	
27	30	.	.	.	.	.	.	

31

MAY

Wk. 22.151-214

SATURDAY

31.05.2014

Case 1: neuron j is output

→ we have desired output then we can use equation (16) directly for local gradient, and using this gradient, we can compute for updating the weights.

Case 2: neuron j is a hidden node.

→ There is no specified desired output for that neuron.

→ error signal for hidden neuron would have to be determined recursively and working backwards.

→ Equation (16) (4.16) we may redefine local gradient $\delta_j(n)$ for hidden neuron as

$$\delta_j(n) = - \frac{\partial E(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial \phi_j(n)}$$

01 Jun • Sun

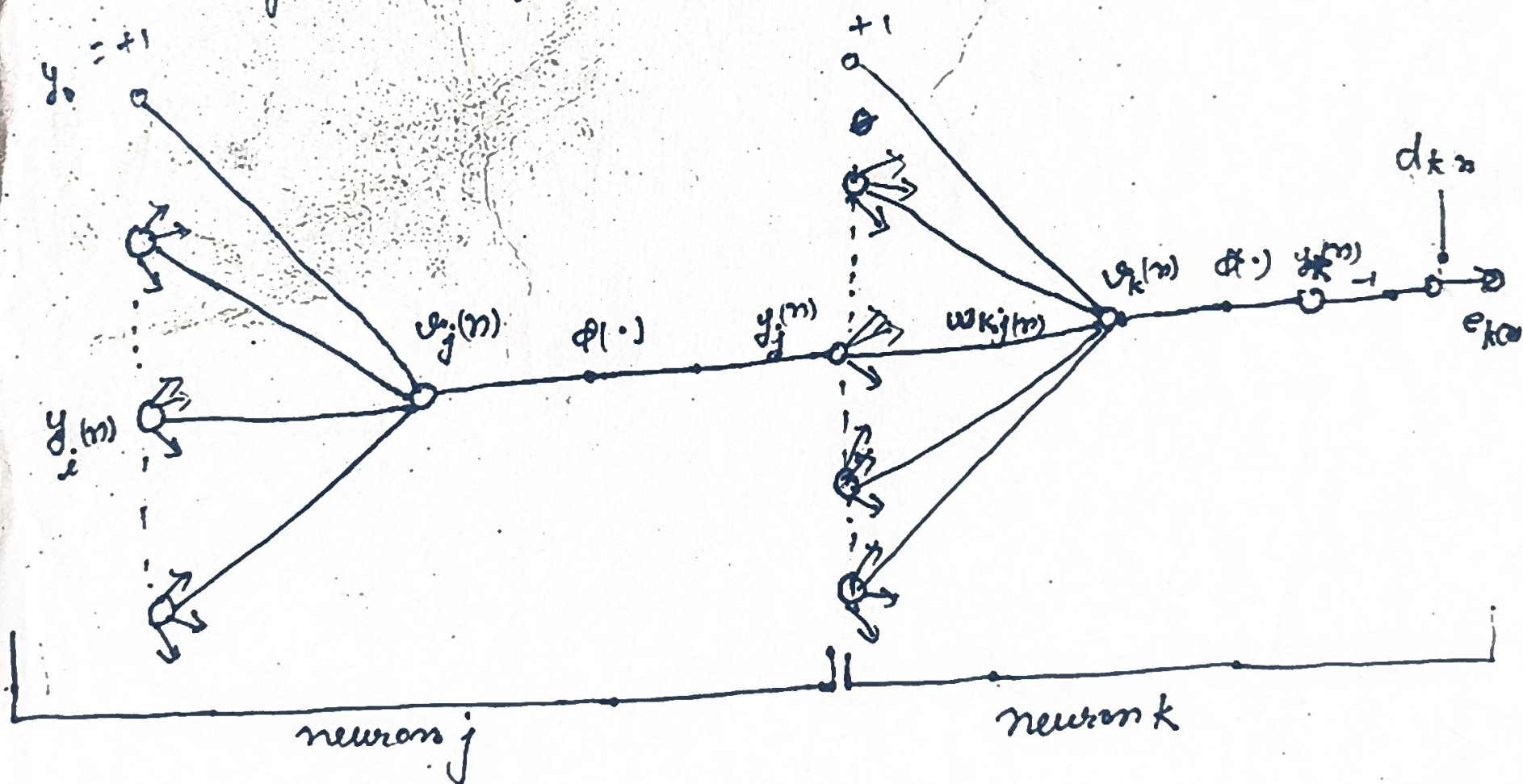
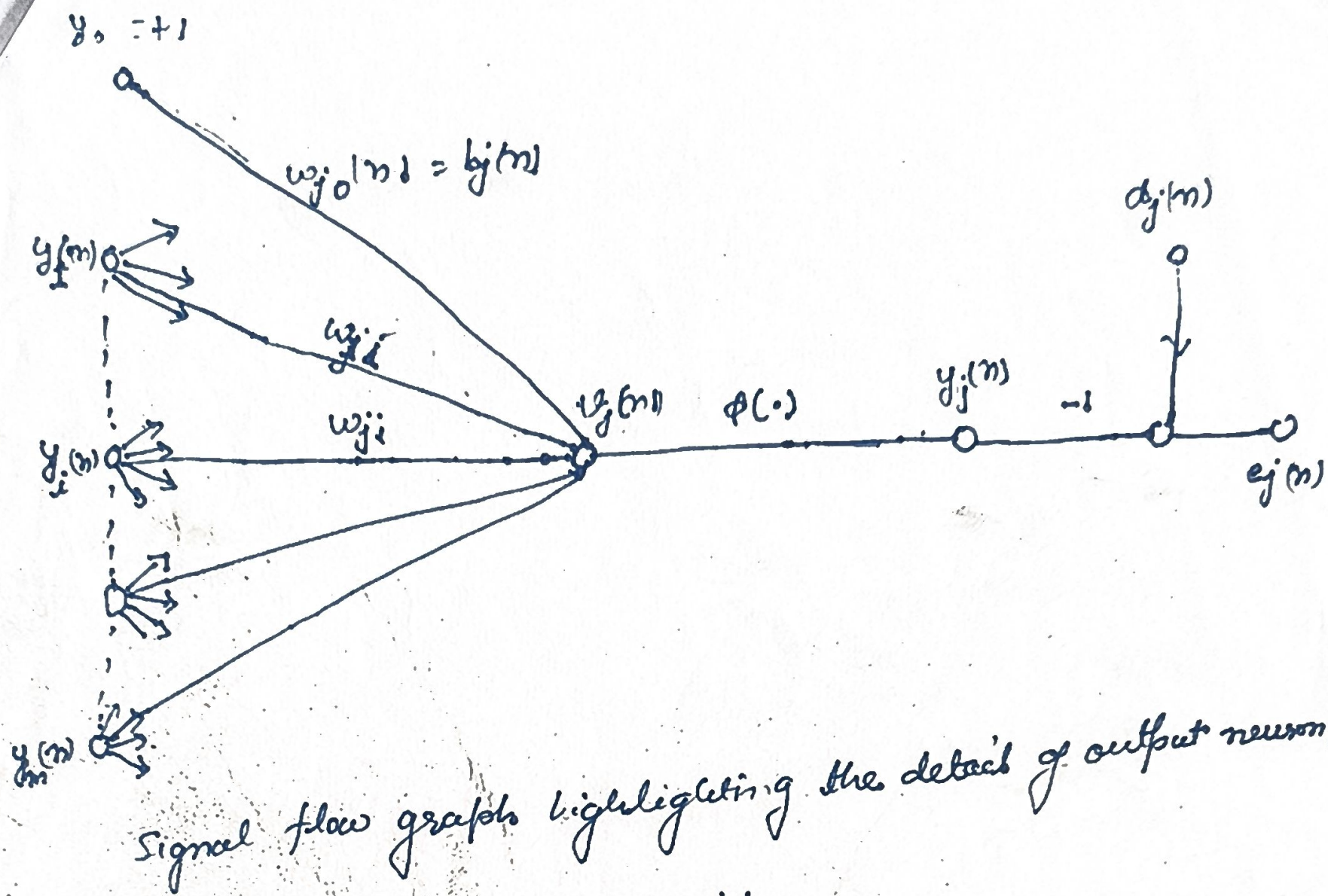
$$= - \frac{\partial E(n)}{\partial y_j(n)} \cdot \phi_j'(\phi_j(n))$$

from equation (17)

MAY

S M T W T F S

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31



Signal - flow graph highlighting the details of output neuron k connected to hidden neuron j